

Anhang: Designentscheidungen

Dieser Anhang dokumentiert die zentralen technischen Designentscheidungen des Geef.Sdk, einschließlich verworfener Alternativen und der jeweiligen Begründung.

1. Immutable Context vs. Mutable State

Entscheidung

`IRunContext` ist vollständig immutabel. Jeder `Set()`-Aufruf erzeugt einen neuen Snapshot auf Basis einer `ImmutableDictionary<string, object>`.

Alternativen

Alternative	Pro	Contra
Mutable Dictionary + Locks	Performanter (keine Kopien)	Thread-Safety-Probleme bei parallelen Reviewern; schwer zu debuggen; keine Snapshot-Historie
Copy-on-Write mit Cow-Wrapper	Lazy Copies, performant bei wenig Writes	Komplexere Implementierung; Copy-Semantik nicht offensichtlich
Event-Sourcing-basierter Context	Vollständige Änderungshistorie	Over-Engineering für den Anwendungsfall; hoher Speicherverbrauch

Begründung

Immutabilität ist das stärkste Architekturprinzip des SDK. Sie ermöglicht:

- **Parallele Reviewer** ohne Synchronisation
- **Deterministische Tests** ohne Setup/Teardown
- **Audit-Trail** durch Snapshot-Vergleiche
- **Kein defensives Kopieren** — Caller können Referenzen frei teilen

Die `ImmutableDictionary` aus `System.Collections.Immutable` verwendet intern ein **balanced tree** (AVL), das strukturelles Sharing bietet. Damit ist `Set()` in $O(\log n)$ statt $O(n)$ — der Overhead gegenüber einem mutablen Dictionary ist marginal.

2. Typisierte Keys vs. String-basierte Keys

Entscheidung

`ContextKey<T>` — ein generischer Record mit Name und Typ.

Alternativen

Alternative	Pro	Contra
Plain Strings	Einfacher, keine Wrapper	Keine Typsicherheit; Tippfehler erst zur Laufzeit
Enum-basierte Keys	IDE-Autocomplete	Nicht erweiterbar; kein generischer Typ
Marker Interfaces	Compile-time Safety	Boilerplate; ein Interface pro Key

Begründung

`ContextKey<T>` bietet Typsicherheit zur Kompilierzeit bei minimalem Boilerplate:

```
var key = new ContextKey<string>("my:data");  
context.Set(key, 42); // Kompilierfehler! int ≠ string
```

Der Name-basierte Lookup im Dictionary bleibt performant (String-Hashing), während der generische Typ den Cast zur Kompilierzeit validiert.

3. Finding-Fingerprints vs. Finding-Equality

Entscheidung

Jeder Finding hat ein explizites `Fingerprint`-Property (String), das vom Reviewer gesetzt wird.

Alternativen

Alternative	Pro	Contra
GetHashCode() / Equals()	Automatisch, kein manueller Aufwand	Hashes brechen bei Nachrichtenänderungen; nicht stabil über Versionen
UUID pro Finding	Einfach, garantiert eindeutig	Keine Stagnations-/Regressionserkennung möglich
Content-Hash (SHA256)	Deterministisch, automatisch	Zu granular — minimale Wortänderungen erzeugen neuen Hash

Begründung

Der Fingerprint muss **semantisch stabil** sein — derselbe logische Fehler muss in verschiedenen Iterationen den gleichen Fingerprint erzeugen, auch wenn die Formulierung leicht variiert. Das kann nur der Reviewer selbst beurteilen:

```
Fingerprint = $"{Name}:{Category}:{NormalizeMessage(msg)}"
```

Automatische Hashes sind entweder zu instabil (jede Wortänderung = neuer Hash) oder zu stabil (verschiedene Fehler kollidieren).

4. Evaluation Strategy als Strategie-Pattern vs. Konfigurationsflag

Entscheidung

`IEvaluationStrategy` als Interface mit vier Implementierungen.

Alternativen

Alternative	Pro	Contra
Enum + switch	Weniger Klassen	Nicht erweiterbar; monolithisch
Builder-Flags (<code>RunInParallel = true</code>)	Einfache API	Kombinatorische Explosion bei mehreren Flags
Decorator-Pattern	Komposierbar	Over-Engineering; Strategie ist atomar, nicht schachtelbar

Begründung

Das Strategy-Pattern ermöglicht:

- **Custom-Strategien** ohne Modification des SDK
 - **Klare Semantik** — jede Strategie hat einen präzisen Vertrag
 - **Testbarkeit** — jede Strategie isoliert testbar
-

5. Middleware-Chain vs. Event-Hooks

Entscheidung

ASP.NET-Core-artige Middleware-Pipeline mit `Func<Task> next`.

Alternativen

Alternative	Pro	Contra
Nur Events	Einfacher, kein Chain-Building	Können Pipeline nicht abbrechen/modifizieren
AOP (Aspect-Oriented)	Transparent, kein expliziter Code	.NET-AOP-Support begrenzt; schwer zu debuggen
Filter-Pipeline (MVC-artig)	Bekanntes Pattern	Zu komplex für 4 Phasen

Begründung

Middleware kann sowohl **beobachten** als auch **eingreifen**:

- `TimeoutMiddleware` bricht die Phase ab und wirft `PhaseTimeoutException`
- `TracingMiddleware` beobachtet nur (Start/Stop eines Spans)
- Eigene Middleware kann den Cancellation-Token ersetzen, Metriken erfassen, Circuit-Breaking implementieren

Events (`IGeefEventSink`) existieren parallel für reine Beobachtung ohne Eingriffsmöglichkeit.

6. Cancellation-Token-Propagation in Middleware

Entscheidung

`GeefMiddlewareContext.Cancellation-Token` ist ein **settable Property** (nicht `init`-only). `TimeoutMiddleware` ersetzt den Token; die Operation-Lambda liest `ctx.Cancellation-Token` zur Ausführungszeit.

Alternativen

Alternative	Pro	Contra
<code>init</code> -only Property	Immutabel, sicherer	Middleware kann Token nicht propagieren; Timeout unwirksam
Token als Lambda-Parameter	Explizit, kein Mutable State	Signatur-Änderung in <code>Func<Task> next</code> ; nicht rückwärtskompatibel
Cancellation-Token-Source als Context-Property	Middleware erstellt linked CTS	Lifecycle-Management komplex; wer disposed?

Begründung

Die Operation-Lambdas in `GeefPipelineRunner` schließen über `ctx` (eine Klasse, kein Struct). Wenn `TimeoutMiddleware` vor dem Aufruf von `next()` den Token ersetzt, sehen alle nachfolgenden

Middleware und die Operation den neuen Token — weil die Lambda den Token **zur Ausführungszeit** aus `ctx` liest, nicht zur Definition.

```
// TimeoutMiddleware:
ctx.CancellationToken = cts.Token; // Ersetzt den Token
await next();                      // Operation liest ctx.CancellationToken

// GeefPipelineRunner (Lambda):
() => _grounding.RunAsync(input, ctx.CancellationToken) // Liest zur Ausführungszeit
```

Das `finally`-Block in `TimeoutMiddleware` stellt den ursprünglichen Token wieder her — wichtig, wenn mehrere Middleware den Token modifizieren.

7. FailFast: Sofortige Rückkehr vs. Task-Draining

Entscheidung

Sofortige Rückkehr nach dem ersten blockierenden Ergebnis. Verbleibende Tasks werden per `ContinueWith(OnlyOnFaulted)` non-blocking beobachtet.

Alternativen

Alternative	Pro	Contra
<code>await Task.WhenAll</code> nach Cancel	Alle Exceptions beobachtet	Blockiert bis alle Tasks fertig — nicht fail-fast
<code>foreach (await remaining)</code>	Sequential Drain, beobachtet	Blockiert — identisches Problem
Fire-and-Forget (keine Observation)	Sofortige Rückkehr	Unobserved Task Exceptions in .NET

Begründung

„Fail-Fast“ bedeutet: **sofortige Rückkehr an den Caller**. Jede Form von `await` auf verbleibende Tasks verletzt diese Semantik.

Die `ContinueWith`-Lösung:

1. Ist non-blocking (keine Rückkehr-Verzögerung)
2. Beobachtet Exceptions (kein `UnobservedTaskException`)
3. Cancellation-Signal ist bereits gesendet (`cts.Cancel()`)
4. Tasks mit kooperativem Cancellation beenden sich zeitnah

8. Keine LLM-SDK-Abhängigkeit

Entscheidung

Das SDK hat **keine Abhängigkeit** auf OpenAI, Anthropic, Azure AI oder andere LLM-SDKs.

Alternativen

Alternative	Pro	Contra
OpenAI-SDK als Peer Dependency	Convenience-Methoden möglich	Lock-in; nicht alle Nutzer verwenden OpenAI
Abstraktes LLM-Interface	Erweiterbar	Yet Another Abstraction; eigene Komplexität
Plugin-System	Maximal flexibel	Over-Engineering für v1

Begründung

Die vier Provider-Interfaces (`IGroundingStep` , `IExecutionStep` , `IReviewer` , `IFinalizer<T>`) sind die Abstraktionsschicht. Nutzer implementieren diese mit ihrem bevorzugten LLM-Client:

```
public class OpenAiExecution(OpenAIClient client) : IExecutionStep
{
    public async Task<ExecutionResult> RunAsync(IRunContext ctx, CancellationToken c
t)
    {
        var response = await client.GetChatCompletionsAsync(..., ct);
        return new ExecutionResult { UpdatedContext = ctx.Set(key, response) };
    }
}
```

Das SDK orchestriert. Der Nutzer integriert. Keine unnötige Kopplung.

9. `GeefPipelineRunner` als Sealed Class vs. Interface

Entscheidung

`GeefPipelineRunner<TOutput>` ist eine `sealed class` , kein Interface.

Alternativen

Alternative	Pro	Contra
<code>IGeefPipeline<T></code> Interface	Mockbar in Consumer-Tests	Zusätzliche Indirektion; Builder produziert ohnehin konkreten Typ
Abstract Base Class	Erweiterbar	GEEF-Loop ist fix, Erweiterung nur über Policies/Strategies

Begründung

Der Runner ist der **Orchestrator**, nicht ein ersetzbarer Service. Seine Logik (Loop, Convergence-Check, Event-Publishing) ist das Kernstück des SDK. Erweiterbarkeit geschieht über:

- `IConvergencePolicy` — Loop-Steuerung
- `IEvaluationStrategy` — Reviewer-Ausführung
- `IGeefMiddleware` — Cross-Cutting Concerns
- `IGeefEventSink` — Beobachtung

Der Runner selbst hat keine variablen Teile. Ein Interface würde suggerieren, dass der Loop selbst austauschbar ist — das widerspricht dem Pattern.

10. Record Types für Results vs. Klassen

Entscheidung

Alle Result-Typen (`GroundingResult`, `ExecutionResult`, `ReviewResult`, `Finding`, etc.) sind `sealed record`.

Begründung

Records bieten:

- **Value Equality** — Vergleich über Properties, nicht Referenz
- `with` -Expressions — Einfaches Klonen mit Änderungen
- **Immutability by Convention** — `init` -only Properties
- **ToString()** — Automatisch lesbare Darstellung für Debugging
- **Deconstruction** — Pattern Matching in switch-Expressions

```
var updated = result with { Decision = ReviewDecision.Approved };
```

Dieser Anhang dient als Referenz für Architektur-Reviews und als Grundlage für Weiterentwicklungen des SDK.